PROGRAMACIÓN WEB I · 4º GRADO EN INGENIERÍA INFORMÁTICA · CURSO 2026-27

Práctica 3 — Cuánto pesa un npm install

Auditoría de las dependencias que mete un asistente de código.

Sesión: 10 (jueves 8 de octubre) · **Modalidad:** individual · **Régimen de IA: D** · **Entrega:** campus virtual, antes de la sesión 11 (jueves 15 de octubre; el lunes 12 es fiesta)

Qué se pide

> **Le vas a pedir a un asistente que convierta una API pequeña a Express y le añada cinco cosas. Después vas a averiguar qué ha metido en tu proyecto, cuánto pesa, y qué hacía falta de verdad.**
>
> Lo que se evalúa es la auditoría de las dependencias, no el código. Que el asistente instale muchos paquetes, o ninguno, es material de trabajo. Lo que se corrige es si sabes qué ha entrado, de dónde viene y si podías haberlo evitado.

Una fila que diga «no sé si este paquete hace falta, y esto es lo que probé» vale más que un «todo necesario» sin datos. No arregles nada antes de auditar: la radiografía «antes» tiene que ser la del proyecto tal como lo dejó el asistente.

Régimen D — lo que significa aquí

	Régimen B	Régimen D (esta práctica)
La IA es...	un medio para hacer el trabajo	**el objeto que se evalúa**
Se evalúa...	el resultado	**tu análisis del resultado**
Trazabilidad	recomendada	**obligatoria**
Declaraciones	autoría + uso de IA	autoría + uso de IA

Sin la declaración de autoría y la declaración de uso de IA la entrega no se corrige.

El material de partida

Descarga practica-sesion10.zip del campus y descomprímelo. Necesitas Node 22 o posterior (node --version) y conexión a internet: npm descarga los paquetes del registro.

Fichero	Qué es
servidor.js	Una API de productos hecha solo con el módulo http de Node, sin ninguna dependencia. GET /productos, GET /productos/:id y POST /productos. Funciona.
servidor.test.cjs	Doce tests que arrancan node servidor.js con PORT=3310 y le hacen peticiones de verdad con fetch. No miran si dentro hay http o Express: solo lo que responde. No se tocan.
revisar.js	La radiografía del proyecto: scripts, dependencias, cuántos paquetes arrastra cada una y si .gitignore excluye node_modules. No opina: solo mide.
AUDITORIA.md	La plantilla de lo que entregas, con los siete apartados.

Fichero	Qué es
LEEME.md	Resumen de los pasos y los comandos.

Los seis pasos

Paso 1 · Comprueba el punto de partida

```
node --test
node revisar.js
```

Tienen que salir 11 tests en verde y 1 en rojo: el 6, porque los ids todavía son números y no UUID. Es lo esperado. revisar.js dirá que no hay package.json: todavía no es un proyecto npm.

Paso 2 · Pídeselo al asistente

El que uses normalmente, en una conversación nueva. Pega servidor.js entero y, a continuación, este prompt. Literal:

> *Tengo esta API hecha con el módulo http de Node (te pego servidor.js). Conviértela en un proyecto npm que use Express, con su package.json. Además:*
>
> *1. Cada producto nuevo tiene que llevar como id un identificador único universal (UUID), no un número.*
>
> *2. El puerto tiene que poder configurarse en un fichero .env.*
>
> *3. Mientras desarrollo, quiero que el servidor se reinicie solo cada vez que guardo un fichero.*
>
> *4. Cada petición tiene que escribirse en la consola con la fecha y la hora en formato dd/mm/aaaa hh:mm:ss.*
>
> *5. Valida bien los datos que llegan en el POST.*
>
> *6. Quiero lanzar los tests con npm test.*
>
> *El fichero principal tiene que seguir llamándose servidor.js, porque los tests lo arrancan así.*

No añadas nada: ni «usa pocas dependencias», ni «no instales paquetes». Queremos ver qué hace él solo. Si te hace preguntas, contesta lo mínimo («como te parezca mejor»). Guarda la conversación entera: es la trazabilidad obligatoria del régimen D.

Paso 3 · Haz exactamente lo que te diga

Crea los ficheros que te dé, con su contenido, y ejecuta sus npm install tal cual. Cuando termines, guarda una copia de su package.json:

```
cp package.json package.asistente.json          # en Windows: copy
```

Si el servidor no arranca o algún test se pone rojo, no lo arregles todavía: apúntalo. Es parte de lo que se audita.

Paso 4 · La radiografía «antes»

```
npm test          # o node --test, si no ha creado el script
node revisar.js
```

Copia las dos salidas enteras en el apartado 2 de AUDITORIA.md.

Paso 5 · Audita cada dependencia

Todas, las de dependencies y las de devDependencies. Para cada una, las cinco preguntas de la clase:

1. ¿Existe, y es el que creo? El nombre exacto, quién lo publica, el repositorio.
2. ¿Lo hace ya Node, o algo que ya tengo? Busca primero en la documentación de Node (nodejs.org/api) y en la de Express (expressjs.com), no en un buscador ni en el asistente.
3. ¿Cuánto trae? El número de paquetes que arrastra, según revisar.js.
4. ¿Está vivo? Última versión, descargas por semana, si está marcado como obsoleto.
5. ¿Qué dice npm audit?

Y una más, que no es de un paquete sino del código: ¿el código que ha escrito el asistente es para la versión mayor que se ha instalado? Mira qué versión de Express hay en package-lock.json y compárala con lo que dice su documentación.

```
npm view <paquete>              # ¿existe? versiones, mantenedores, repositorio
npm ls --all                    # el árbol entero
npm explain <paquete>           # ¿quién lo ha traído?
npm audit                       # vulnerabilidades conocidas
```

Las descargas por semana y la fecha de la última versión están en la página de cada paquete en npmjs.com.

Paso 6 · Déjalo con lo mínimo

Quita lo que sobre (npm uninstall paquete) y cubre cada requisito con lo que ya tienes. Al final:

- Los seis requisitos del prompt se siguen cumpliendo. Demuestra cada uno: por ejemplo, cambia el puerto en .env y enseña que el servidor arranca en el nuevo.
- npm test da 12 en verde, sin haber tocado servidor.test.cjs.
- node revisar.js da la radiografía «después». Cópiala en el apartado 6.

Si te da tiempo: compáralo con un compañero

Mismo prompt, asistentes distintos, dependencias distintas. Si el tuyo metió cosas que el suyo no, o al revés, anótalo en la auditoría: dice mucho de cómo funcionan estas herramientas.

Qué se entrega

Un ZIP con tu proyecto final y la auditoría.

Qué	Detalle
servidor.js, package.json, package-lock.json, .gitignore	El proyecto final, con lo mínimo y los 12 tests en verde.
package.asistente.json	El package.json tal como lo dejó el asistente, del paso 3.
.env.ejemplo	Un .env de muestra, sin datos reales. El .env de verdad no se entrega nunca.
AUDITORIA (md, pdf o docx)	La plantilla rellena, con sus siete apartados.
node_modules	NO. Se reconstruye con npm install a partir de package.json y package-lock.json.

La tabla del apartado 3

Una fila por dependencia:

Paquete	Pedida → instalada	¿Para qué?	¿Existe? · descargas · última	Arrastra	¿Ya lo hace Node o Express?	Veredicto
express	^5.x → 5.x.x	Pedido	Sí · … · …	…	No: es lo que se pide	**Imprescindible**
…						

Veredictos: Imprescindible · Sustituible (di con qué) · Sobra · Sospechosa (no existe, abandonada, nombre raro). La primera fila es un ejemplo de cómo se rellena.

Cómo se evalúa

Qué se mira	Peso
La tabla: cada dependencia con datos comprobados y un veredicto justificado	**35 %**
Las sustituciones: qué quitaste, con qué lo cubriste y cómo comprobaste que el requisito se cumple	**20 %**
Las radiografías antes y después, y qué significa la diferencia	**10 %**
Las versiones: rangos, versión instalada, el lock, y si el código encaja con la versión mayor instalada	**10 %**
El proyecto final: 12 en verde sin tocar los tests, y sin node_modules en el ZIP	**10 %**
Trazabilidad: conversación completa, package.asistente.json, herramienta y modelo	**15 %**

Lo que NO baja la nota

Que el asistente instale muchos paquetes. Que no instale ninguno. Que su código no arranque. Que una fila diga «no estoy seguro», si explicas qué probaste. Nada de eso es culpa tuya.

Lo que SÍ la baja

Un veredicto sin datos. Entregar node_modules o el .env. Modificar los tests. Añadir instrucciones al prompt. Entregar sin declaraciones.

La regla que sale de esta práctica

La mejor dependencia es la que no instalas. La segunda mejor, la que has auditado.

En la sesión 11 la corregimos juntos, y usamos vuestros proyectos para aprender a depurar un servidor Express.